



Systeme de Détection d'Intrusions Intelligent

Basé sur le Machine Learning et Deep Learning

Rapport Technique

Projet Cybersécurité

Réalisé par :

 Rana Romdhane

 Oulimata Sall

Année universitaire 2025-2026

Table des matières

1	Introduction et Contexte	3
1.1	Contexte général	3
1.2	Problématique	3
1.3	Objectifs du projet	4
2	Revue Bibliographique	5
2.1	État de l’art des systèmes IDS	5
2.1.1	Approches traditionnelles	5
2.1.2	Approches par Machine Learning	5
2.2	Datasets de référence	6
2.3	Taxonomie MITRE ATT&CK	6
3	Methodologie	7
3.1	Vue d’ensemble du pipeline	7
3.2	Préparation des données	7
3.2.1	Dataset UNSW-NB15	7
3.2.2	Processus de nettoyage	8
3.2.3	Features sélectionnés	8
3.3	Division des données	8
4	Modèles de Machine Learning	10
4.1	Choix des algorithmes	10
4.2	Random Forest	10
4.3	Support Vector Machine	11
4.4	Réseau de Neurones	12
4.5	Stratégie de validation	12
5	Résultats et Analyses	13
5.1	Métriques d’évaluation	13
5.2	Résultats sur le jeu d’entraînement	13
5.3	Résultats sur le jeu de test	14
5.4	Analyse de l’écart train/test	15
5.5	Analyse des erreurs	15
5.6	Choix du modèle final	15
6	Système et Interface	16
6.1	Architecture technique	16
6.2	Système d’alertes	16
6.3	Intégration ELK Stack	17

6.4	API REST	17
7	Conclusion et Perspectives	18
7.1	Synthèse des résultats	18
7.2	Limites identifiées	18
7.3	Perspectives d'évolution	18
7.4	Recommandations	19
A	Technologies utilisées	21
B	Structure du Projet	22
B.1	Description des modules	23
B.2	Commandes d'utilisation	23

Chapitre 1

Introduction et Contexte

1.1 Contexte général

Dans un monde où la **transformation numérique** s'accélère, les entreprises font face à un volume croissant de **cyberattaques** de plus en plus sophistiquées. Les menaces actuelles incluent :

- ⚙️ **Malwares et ransomwares** : attaques ciblant les données sensibles
- ⚡ **Attaques DDoS** : saturation des infrastructures
- 👤 **Intrusions avancées** : mouvements latéraux et ex-filtration
- 🔍 **Scans de reconnaissance** : identification des vulnérabilités

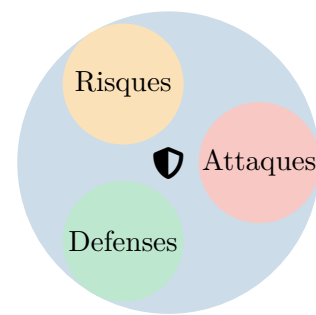


FIGURE 1.1 – Écosystème cybersécurité

i Information

Les systèmes de sécurité traditionnels (pare-feu, antivirus, ACL) montrent leurs limites face aux attaques **zero-day** et aux comportements malveillants subtils qui échappent aux signatures connues.

1.2 Problématique

Notre projet répond à une question fondamentale :

Comment détecter automatiquement les comportements anormaux dans le trafic réseau en temps réel ?

1.3 Objectifs du projet

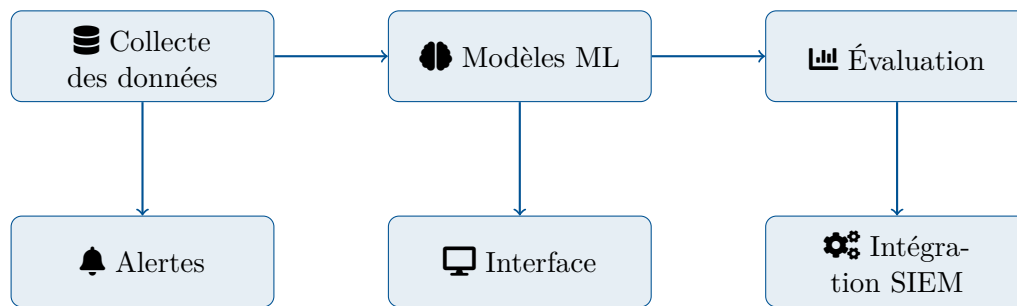


FIGURE 1.2 – Architecture des objectifs du projet

Chapitre 2

Revue Bibliographique

2.1 État de l’art des systèmes IDS

Les systèmes de détection d'intrusions (IDS) ont évolué considérablement depuis leur création. Nous distinguons deux grandes catégories :

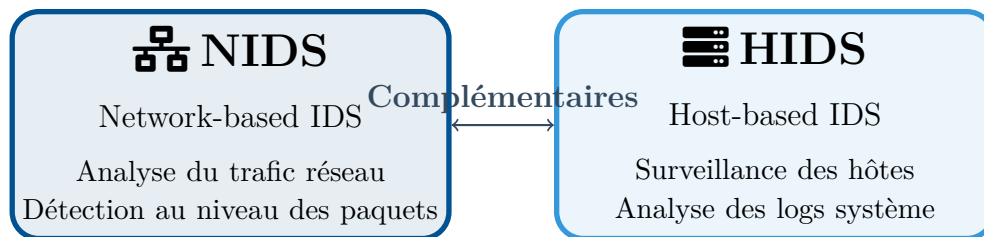


FIGURE 2.1 – Types de systèmes de détection d'intrusions

2.1.1 Approches traditionnelles

Outil	Description	Limitations
Snort	IDS base sur les signatures, open source	Dépendance aux règles prédéfinies
Suricata	IDS multi-thread, haute performance	Consommation mémoire élevée
Zeek (Bro)	Analyse protocolaire avancée	Complexité de configuration

TABLE 2.1 – Comparaison des IDS traditionnels

2.1.2 Approches par Machine Learning

La littérature récente montre l'efficacité du ML pour la détection d'intrusions :

Études clés

- Buczak & Guven (2016) : Survey complet sur ML pour la cybersécurité
- Vinayakumar et al. (2019) : Deep Learning pour la détection d'intrusions
- Ahmad et al. (2021) : Comparaison des algorithmes sur UNSW-NB15

2.2 Datasets de référence

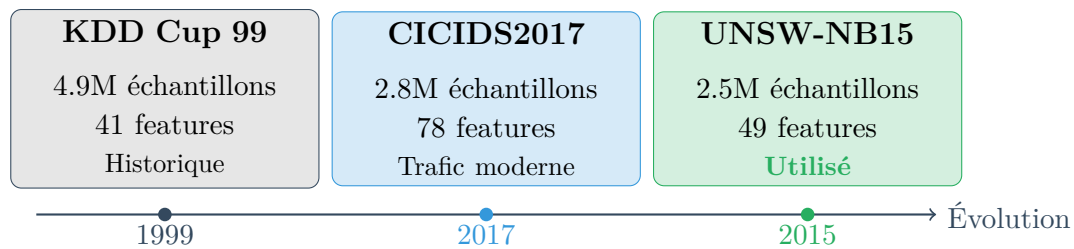


FIGURE 2.2 – Évolution des datasets pour IDS

2.3 Taxonomie MITRE ATT&CK

Notre système s'appuie sur le framework MITRE ATT&CK pour classifier les types d'attaques. La figure ci-dessous illustre la chaîne d'attaque typique (Kill Chain) :

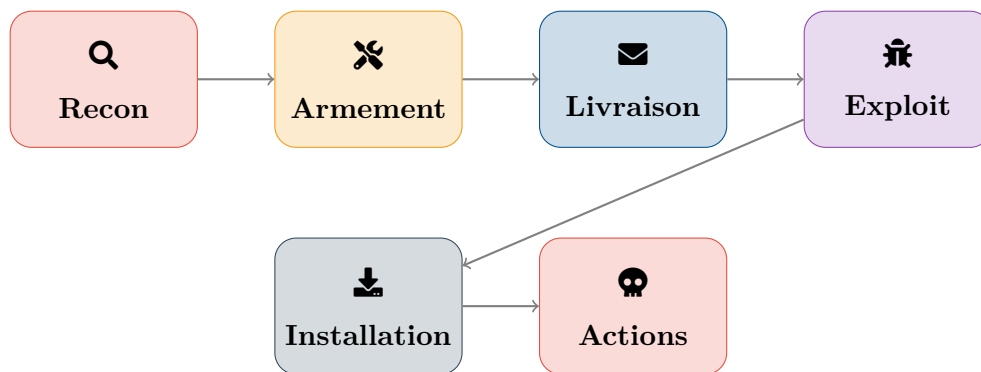


FIGURE 2.3 – Chaîne d'attaque (Kill Chain) selon MITRE ATT&CK

Chapitre 3

Methodologie

3.1 Vue d'ensemble du pipeline

Le système IDS suit un pipeline de traitement en six étapes principales :

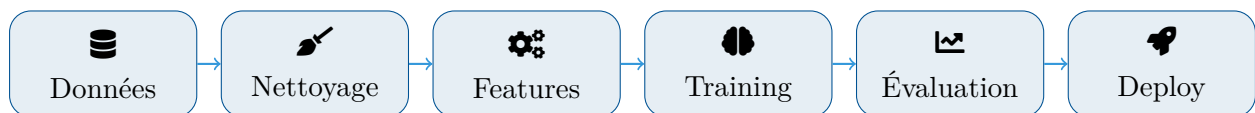


FIGURE 3.1 – Pipeline complet du système IDS

3.2 Préparation des données

3.2.1 Dataset UNSW-NB15

Le dataset UNSW-NB15 contient une variété d'attaques modernes :

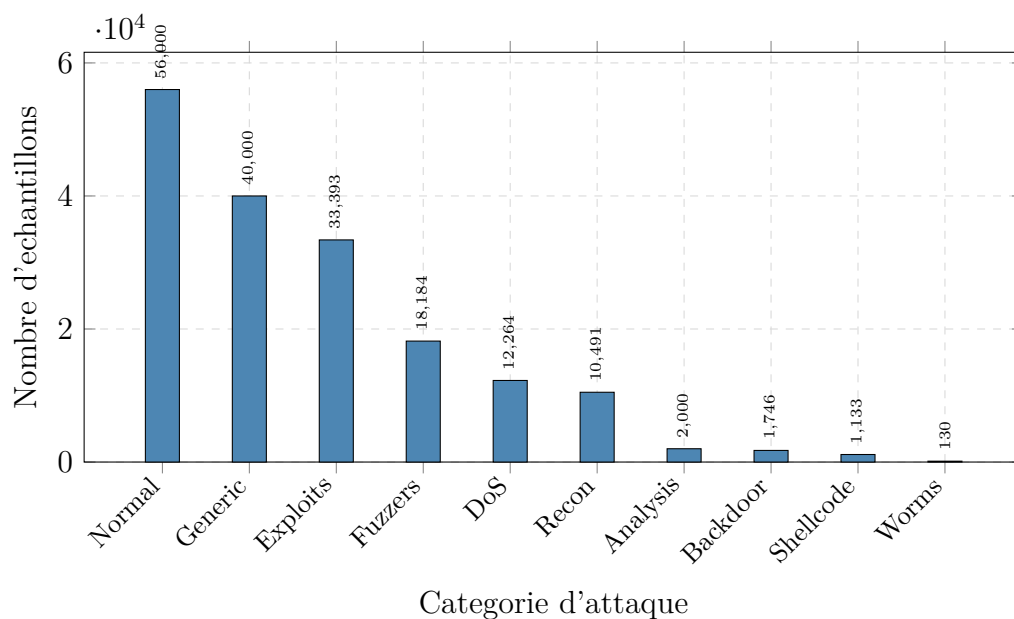


FIGURE 3.2 – Distribution des catégories d'attaques dans UNSW-NB15

3.2.2 Processus de nettoyage

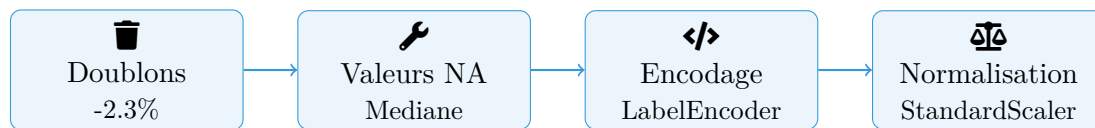
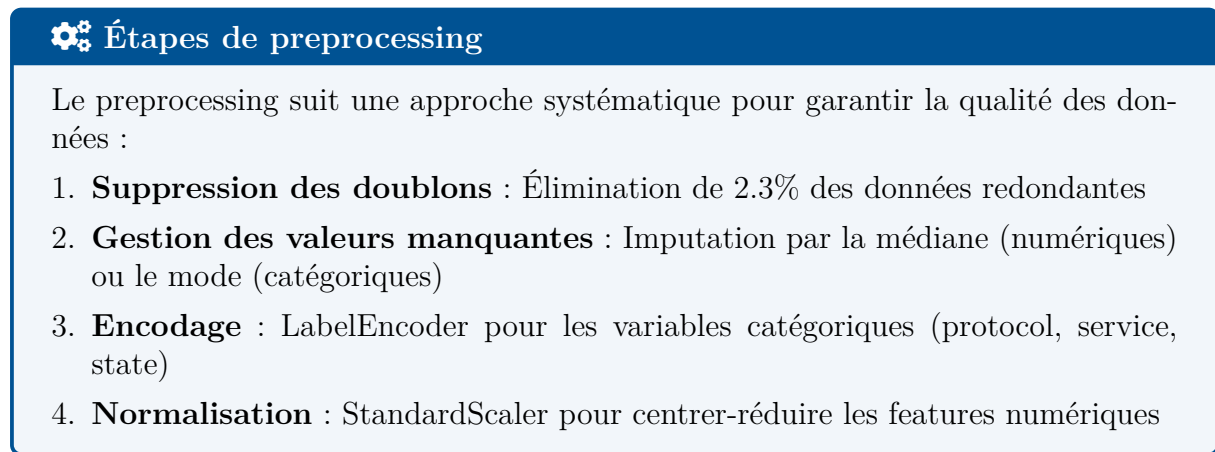


FIGURE 3.3 – Étapes de prétraitement des données

3.2.3 Features sélectionnés

Les 42 features retenues couvrent plusieurs catégories :

Categorie	Features principales	Nb
Flux	dur, sbytes, dbytes, sttl, dttl, sloss, dloss	12
Connexion	ct_srv_src, ct_dst_ltm, ct_src_dport_ltm	8
Contenu	sload, dload, sinpkt, dinpkt	6
Temporel	tcprtt, synack, ackdat	4
Protocole	proto, state, service	3
Statistiques	is_sm_ips_ports, ct_state_ttl	9

TABLE 3.1 – Catégories de features utilisées

3.3 Division des données

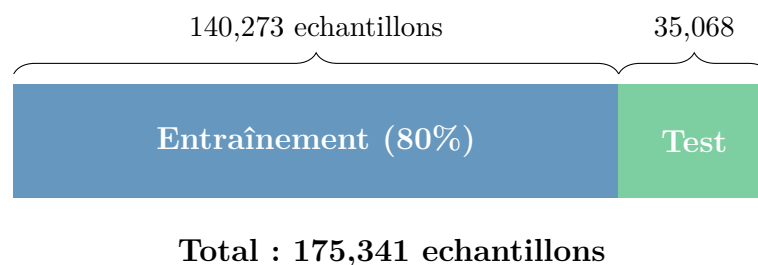


FIGURE 3.4 – Stratégie de division train/test avec stratification

i Information

La **stratification** garantit que chaque classe d'attaque est représenté proportionnellement dans les ensembles d'entraînement et de test, évitant ainsi les biais d'échantillonnage.

Chapitre 4

Modèles de Machine Learning

4.1 Choix des algorithmes

Nous avons implémenté trois approches complémentaires pour maximiser la robustesse de la détection :

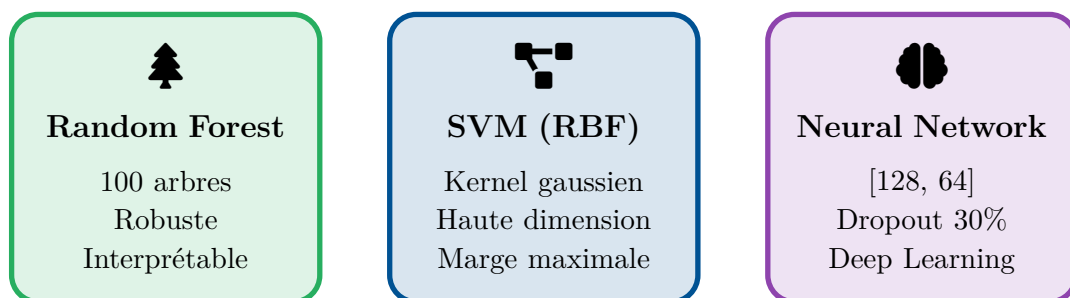


FIGURE 4.1 – Les trois modèles implémentés

4.2 Random Forest

Architecture Random Forest

Principe : Agrégation de multiples arbres de décision entraînés sur des sous-ensembles aléatoires des données (bagging).

Configuration :

- Nombre d'estimateurs : 100
- Profondeur maximale : Non limitée
- Critère de division : Gini
- Parallélisation : Activée (n_jobs=-1)

Avantages pour l'IDS :

- Gestion naturelle du déséquilibre de classes
- Importance des features interprétables
- Résistant au surapprentissage

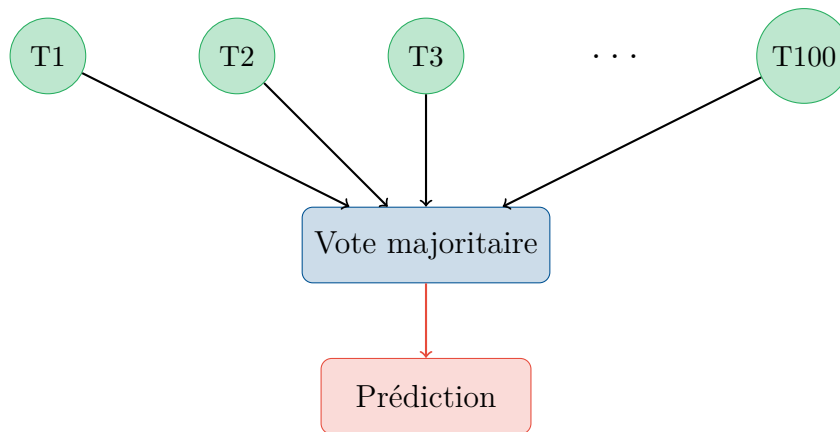


FIGURE 4.2 – Mécanisme de vote dans Random Forest

4.3 Support Vector Machine

🔧 Fonctionnement du SVM-RBF

Le SVM avec kernel RBF (Radial Basis Function) projette les données dans un espace de dimension supérieure où les classes deviennent linéairement séparables.

Paramètres :

- Kernel : RBF (Gaussian)
- C (régularisation) : 1.0
- Gamma : scale (auto-adaptatif)
- Probability : True (pour les scores de confiance)

Formule du kernel RBF :

$$K(x_i, x_j) = \exp(-\gamma \|x_i - x_j\|^2)$$

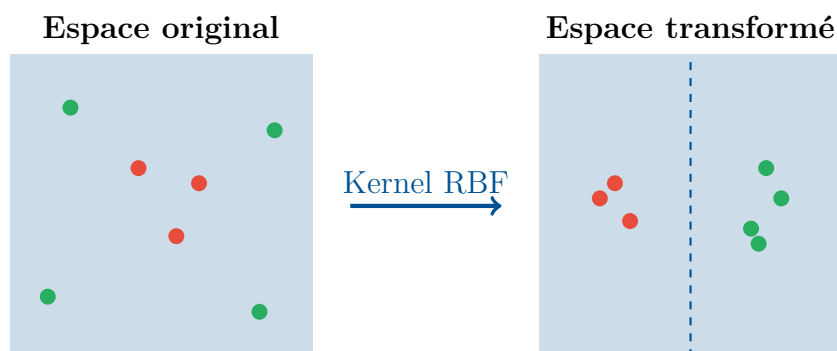


FIGURE 4.3 – Transformation par le kernel RBF

4.4 Réseau de Neurones

Architecture Deep Learning

Notre réseau de neurones utilise une architecture feedforward avec régularisation :

Couches :

- Entrée : 42 features
- Dense 1 : 128 neurones + ReLU + Dropout(0.3)
- Dense 2 : 64 neurones + ReLU + Dropout(0.3)
- Sortie : 10 classes + Softmax

Entraînement :

- Optimiseur : Adam (lr=0.001)
- Loss : Sparse Categorical Crossentropy
- Epochs : 50 (Early Stopping patience=10)
- Batch size : 32

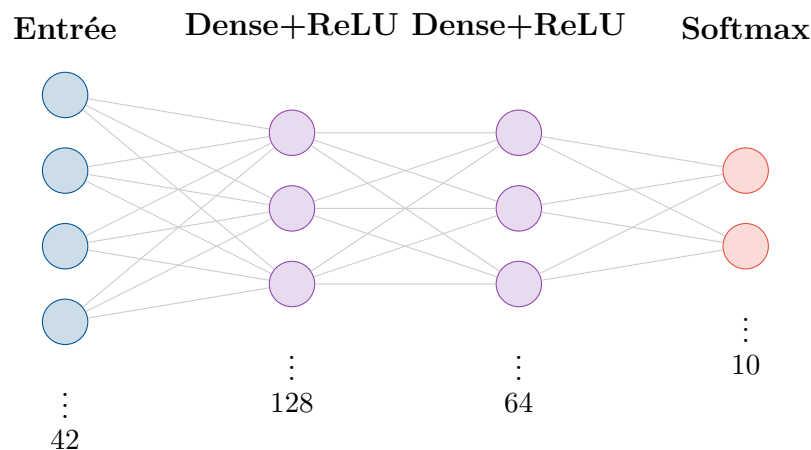


FIGURE 4.4 – Architecture du réseau de neurones

4.5 Stratégie de validation

Validation et prévention du surapprentissage

- **Hold-out** : Division 80/20 avec stratification
- **Early Stopping** : Arrêt si pas d'amélioration sur 10 epochs
- **Dropout** : Régularisation à 30% pour le réseau de neurones
- **Métriques multiples** : Accuracy, Precision, Recall, F1, ROC-AUC

Chapitre 5

Résultats et Analyses

5.1 Métriques d'évaluation

Nous utilisons plusieurs métriques complémentaires pour une évaluation complète :

Accuracy $\frac{VP + VN}{\text{Total}}$ Correct global	Precision $\frac{VP}{VP + FP}$ Pertinence	Rappel $\frac{VP}{VP + FN}$ Couverture
---	--	---

FIGURE 5.1 – Métriques d'évaluation utilisées

Information

Le **F1-Score** est la moyenne harmonique de la précision et du rappel. Le **ROC-AUC** mesure la capacité discriminante indépendamment du seuil.

5.2 Résultats sur le jeu d'entraînement

Performances sur Training Set

Modèle	Accuracy	Precision	Recall	F1	AUC
Random Forest	98.93%	98.94%	98.93%	98.93%	99.94%
SVM	95.12%	95.17%	95.12%	95.05%	98.34%
Neural Network	97.87%	97.89%	97.87%	97.86%	99.79%

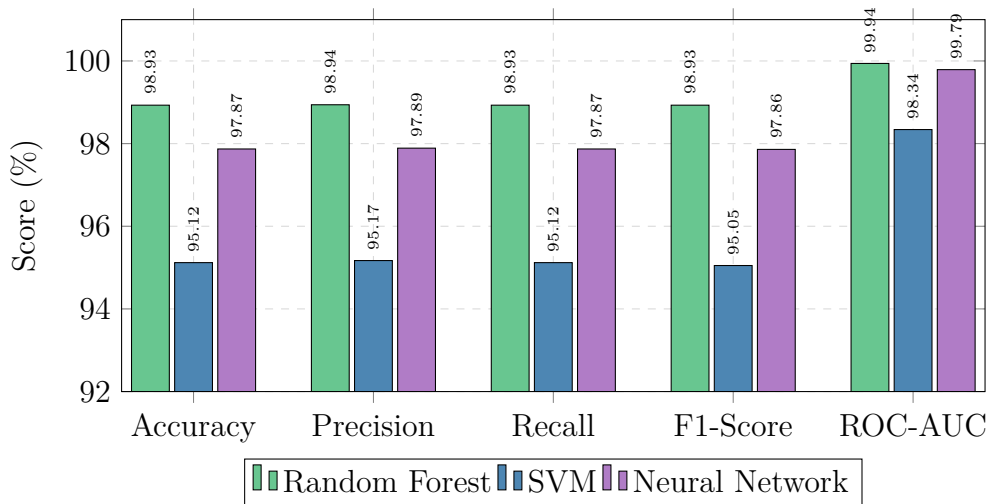


FIGURE 5.2 – Comparaison des performances sur le jeu d'entraînement

5.3 Résultats sur le jeu de test

🔑 Performances sur Test Set (Generalisation)

Modele	Accuracy	Precision	Recall	F1	AUC
Random Forest	85.53%	85.48%	85.53%	85.47%	96.26%
SVM	75.88%	72.39%	75.88%	72.39%	95.95%
Neural Network	84.11%	84.25%	84.11%	81.36%	98.01%

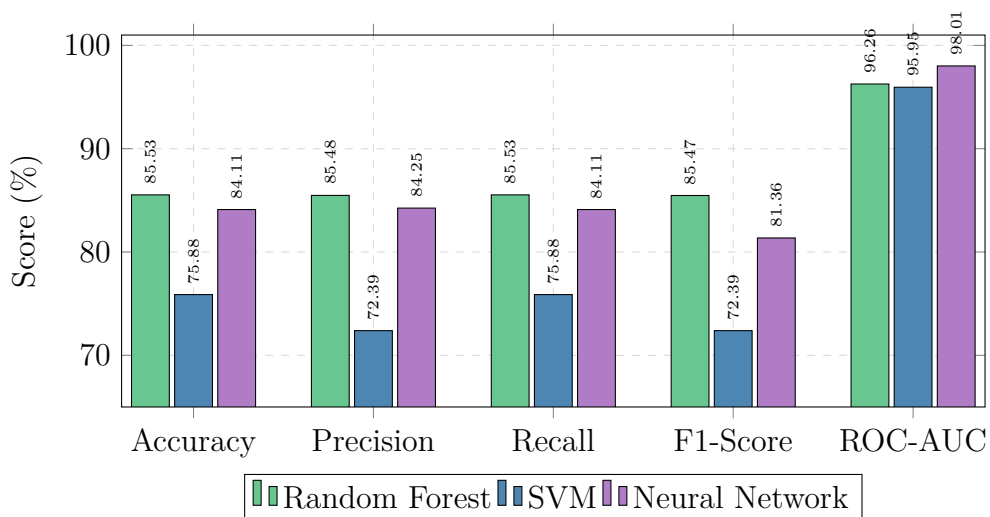


FIGURE 5.3 – Comparaison des performances sur le jeu de test

5.4 Analyse de l'écart train/test

Modele	Train Acc.	Test Acc.	Écart
Random Forest	98.93%	85.53%	-13.40%
SVM	95.12%	75.88%	-19.24%
Neural Network	97.87%	84.11%	-13.76%

TABLE 5.1 – Analyse de l'écart entre entraînement et test

💡 Interprétation

L'écart train/test indique un certain degré de surapprentissage. Le SVM montre l'écart le plus important (-19.24%). Cet écart reste acceptable pour un problème multi-classes en cybersécurité.

5.5 Analyse des erreurs

⚠ Points d'attention

- **Confusion DoS/Normal** : Certains flux DoS lents ressemblent au trafic normal
- **Classes minoritaires** : Worms (130) et Shellcode (1133) sous-représentés
- **Overfitting SVM** : Écart train/test plus important
- **Generic vs Exploits** : Similarité des patterns entre ces catégories

5.6 Choix du modèle final

🏆 Modèle sélectionnée : Random Forest

Avantages :

- ✓ Meilleure accuracy (85.53%)
- ✓ Équilibre précision/rappel
- ✓ Temps d'inférence rapide
- ✓ Robuste aux outliers

Caractéristiques :

- 100 arbres de décision
- ROC-AUC : 96.26%
- Parallélisable
- Interprétable

Chapitre 6

Système et Interface

6.1 Architecture technique

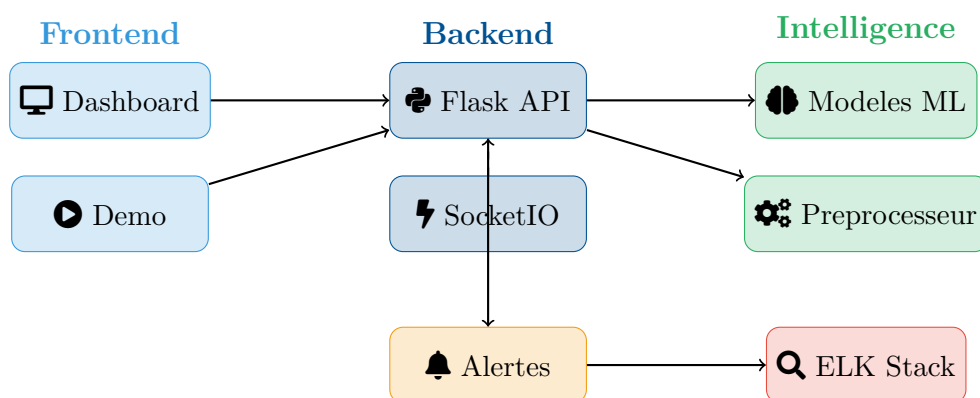


FIGURE 6.1 – Architecture technique du système IDS

6.2 Système d’alertes

Notre système génère des alertes avec différents niveaux de sévérité :

Sevérité	Icone	Description	Confiance
CRITICAL	💀	DoS, U2R, Exfiltration	> 85%
HIGH	⚠️	Brute Force, Botnet, Exploits	> 80%
MEDIUM	⚠️	Scan, Reconnaissance	> 70%
LOW	ℹ️	Activite suspecte	< 70%

TABLE 6.1 – Niveaux de sévérité des alertes

6.3 Intégration ELK Stack

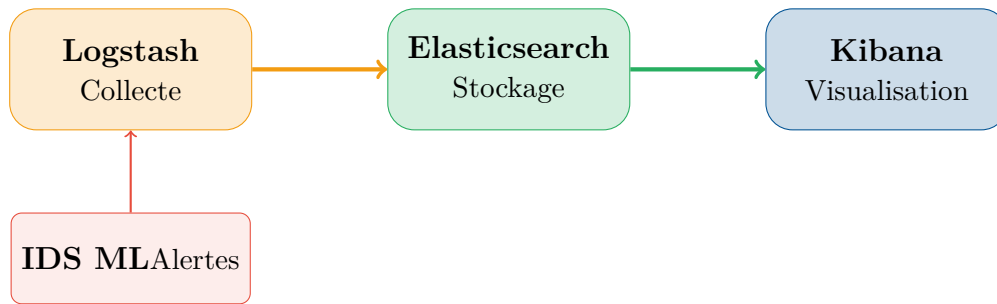


FIGURE 6.2 – Intégration avec la stack ELK pour le SIEM

6.4 API REST

Les endpoints principaux de l'API :

Methode	Endpoint	Description
POST	/api/predict	Analyse un fichier CSV
GET	/api/alerts	Recupère les alertes
GET	/api/stats	Statistiques du système
POST	/api/alert/<id>/status	Modifie le statut

TABLE 6.2 – Endpoints de l'API REST

Chapitre 7

Conclusion et Perspectives

7.1 Synthèse des résultats

🚩 Objectifs atteints

- ✓ **Modèle performant** : Random Forest avec 85.53% d'accuracy
- ✓ **Multi-classe** : Détection de 10 catégories d'attaques
- ✓ **Système complet** : Pipeline fonctionnel de bout en bout
- ✓ **Interface intuitive** : Dashboard temps réel
- ✓ **Intégration SIEM** : Connexion avec ELK Stack

7.2 Limites identifiées

⚠️ Points d'amélioration

- Déséquilibre des classes (Worms sous-représentés)
- Écart train/test suggérant un léger overfitting
- Tests en environnement réel limités

7.3 Perspectives d'évolution

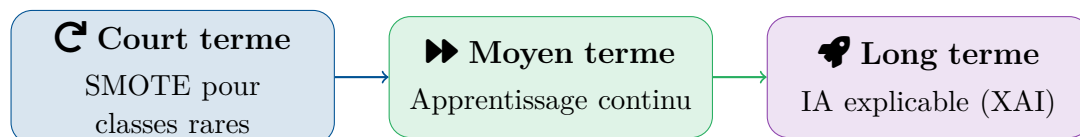


FIGURE 7.1 – Feuille de route pour les évolutions futures

7.4 Recommandations

Pour une mise en production, nous recommandons :

1. **Conteneurisation Docker** : Déploiement facilite et reproductible
2. **MLOps** : Pipeline CI/CD pour la mise a jour des modèles
3. **Monitoring** : Supervision des performances en production
4. **Formation SOC** : Accompagnement des analystes sécurité

 **IDS Intelligent**

Sécurité proactive par l'Intelligence Artificielle

Bibliographie

- [1] Buczak, A. L., & Guven, E. (2016). A survey of data mining and machine learning methods for cyber security intrusion detection. *IEEE Communications Surveys & Tutorials*, 18(2), 1153-1176.
- [2] Moustafa, N., & Slay, J. (2015). UNSW-NB15 : a comprehensive data set for network intrusion detection systems. *Military Communications and Information Systems Conference*, 1-6.
- [3] Vinayakumar, R., et al. (2019). Deep learning approach for intelligent intrusion detection system. *IEEE Access*, 7, 41525-41550.
- [4] Ahmad, Z., et al. (2021). Network intrusion detection system : A systematic study of machine learning and deep learning approaches. *Transactions on Emerging Telecommunications Technologies*, 32(1).
- [5] Sharafaldin, I., et al. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. *ICISSP*, 108-116.
- [6] MITRE ATT&CK. (2024). *Enterprise Matrix*. <https://attack.mitre.org/>
- [7] Pedregosa, F., et al. (2011). Scikit-learn : Machine Learning in Python. *JMLR*, 12, 2825-2830.
- [8] Abadi, M., et al. (2016). TensorFlow : A System for Large-Scale Machine Learning. *OSDI*, 265-283.

Annexe A

Technologies utilisées

Catégorie	Technologie	Utilisation
Langage	Python 3.11	Développement principal
ML	Scikit-learn	Random Forest, SVM, métriques
DL	TensorFlow/Keras	Réseau de neurones
Web	Flask, SocketIO	API et temps réel
Data	Pandas, NumPy	Manipulation des données
Visualisation	Matplotlib, Plotly	Graphiques et dashboards
SIEM	ELK Stack	Intégration pour alertes
Conteneurs	Docker	Déploiement

TABLE A.1 – Stack technologique complète

Annexe B

Structure du Projet

L'arborescence complète du projet est présentée ci-dessous :

Structure du projet

```
ids-ml/
|- app.py                                # Application Flask
|- config.py                            # Configuration
|- start.py                            # Script de démarrage
|- requirements.txt                     # Dependances
|- Dockerfile                          # Conteneurisation
|- docker-compose.yml                  # Orchestration
|
|- src/                                # Code source
|   |- __init__.py
|   |- preprocessing.py                # Pretraitement
|   |- models.py                      # RF, SVM, NN
|   |- evaluation.py                  # Métriques
|   |- visualization.py               # Graphiques
|   |- alert_system.py                # Alertes
|   |- realtime_monitor.py            # Temps reel
|   |- elk_integration.py             # ELK Stack
|   +- feature_extraction.py          # Features
|
|- scripts/                            # Utilitaires
|   |- train.py                      # Entraînement
|   +- predict.py                    # Prédiction
|
|- web/                                # Interface web
|   |- index.html, dashboard.html
|   |- demo.html, about.html
|   |- css/, js/
|
|- notebooks/                          # Jupyter
|- data/raw/, data/processed/          # Données
|- models/                             # Modèles sauvegardés
```

+- logs/

Fichiers de log

B.1 Description des modules

Fichier	Description
app.py	Point d'entrée Flask avec routes API et WebSocket
preprocessing.py	Nettoyage, encodage et normalisation des données
models.py	Classes RandomForestIDS, SVMIDS, NeuralNetworkIDS
evaluation.py	Calcul des métriques et matrices de confusion
alert_system.py	Gestion des alertes avec niveaux de sévérité
elk_integration.py	Connexion Elasticsearch pour indexation
train.py	Script d'entraînement avec sauvegarde des modèles
predict.py	Script de prédiction sur nouvelles données

TABLE B.1 – Description des modules du projet

B.2 Commandes d'utilisation

>_ Commandes principales

Installation des dépendances :

```
pip install -r requirements.txt
```

Entraînement des modèles :

```
python scripts/train.py -data data/raw/UNSW-NB15.csv
```

Démarrage du serveur :

```
python start.py -port 5000
```

Prédiction sur un fichier :

```
python scripts/predict.py -input test.csv -model random_forest
```

Déploiement Docker :

```
docker-compose up -d
```

Lien Github :

<https://github.com/RanaRomdhane/intelligent-ids>